Project

Report

On

Android Password-Based Door Opener

Submitted in partial fulfilment of the requirement for the award of degree of

Bachelors of Technology (B.Tech)

in

Electronics and Communication Engineering

Submitted by: JAGJIT KAUR



Department of Electronics Engineering,

Sri Guru Granth Sahib World University

Fatehgarh Sahib, Punjab, India

ACKNOWLEDGEMENT

The satisfaction and sense of accomplishment that accompany the successful completion of a project would be incomplete without acknowledging the individuals whose guidance, support, and encouragement made it possible. I am pleased to present this project, which represents the integration of academic knowledge, research, and practical implementation.

I express my sincere gratitude to my internal guide, **Dr. Gurjinder Singh, In-Charge of the Department of Electronics Engineering**, for his continuous support, valuable guidance, encouragement, and constructive suggestions throughout the development of this project. His expertise and cooperation have contributed significantly to its successful completion.

Finally, I extend my heartfelt thanks to all faculty members and individuals who directly or indirectly supported and contributed to the completion of this project.

DECLARATION

I, the undersigned, declare that the project entitled ‘Android password based door opener’, being submitted in partial fulfillment for the award of Bachelor of Technology (B.Tech) Degree in Electronics and Communication Engineering, affiliated to Sri Guru Granth Sahib World University, is the work carried out by me.

ABSTRACT

This project presents an Android-based password-controlled door-opening system using an 8051-family microcontroller. The system combines wireless communication, password validation and motor control to demonstrate a practical application of embedded systems in access control.

An Android device transmits commands through Bluetooth to a receiver connected to the microcontroller's serial interface. The embedded C program validates a predefined four-digit password against the received input. When the correct password is entered, the system accepts subsequent door-control commands. The microcontroller then operates a DC motor through an L293D motor-driver circuit to demonstrate opening and closing operations.

An LCD provides information about password entry, validation and door status. The project illustrates the integration of an Android interface, Bluetooth communication, embedded programming and electronic hardware for a remotely controlled door-opening prototype.

LIST OF CONTENTS

Acknowledgement
Declaration
Abstract
List of Contents
List of Figures

Chapter1:	**Embedded System**	**1**
	1.1 Introduction	1
	1.2 Examples	1
	1.3 Features	1
	1.4 Components	2
Chapter2:	**Hardware and Software requirement**	**4**
	2.1 Battery	4
	2.2 Voltage Regulator	5
	2.3 Rectifier	5
	2.4 Filter	6
	2.5 Microcontroller AT89S52	7
	2.6 LED'S	10
	2.7 L293D	11
	2.8 DC Motor	12
	2.9 Factors of ANDROID	14
	2.10 1N4007	20
	2.11 Resistor	21
	2.12 Capacitors	21
	2.13 Software requirements	22
Chapter3:	**Project Working**	**24**
	3.1 Schematic Diagram	24
	3.2 Description	25
	3.3 Project diagram	28
	3.4 Coding	29
	3.5 Hardware Testing	38

CONCLUSION

List of Figures

Figure no.	**Figure Name**	**page no.**
1.1	Embedded system	2
2.1	Voltage Regulator 7805	5
2.2	Rectifier	6
2.3	Filter	6
2.4	Pin configuration of AT89S52	8
2.5	LED	10
2.6	Pin diagram of L293D	11
2.7	DC motor	12
2.8	DC motor	13
2.9	1N4007	21
2.10	Resistor	21
2.11	Capacitor	22
3.1	Schematic Diagram	24
3.2	Microcontroller	27
3.3	Project Diagram	28

CHAPTER: 1

INTRODUCTION TO EMBEDDED SYSTEM

1.1 Introduction:-

An embedded system is a computer system with a dedicated function within a larger mechanical or electrical system, often with real-time computing constraints. It is embedded as part of a complete device often including hardware and mechanical parts. Embedded systems control many devices in common use today. Ninety-eight percent of all microprocessors are manufactured as components of embedded systems.

Modern embedded systems are often based on microcontrollers (i.e. CPU's with integrated memory or peripheral interfaces), but ordinary microprocessors (using external chips for memory and peripheral interface circuits) are also common, especially in more-complex systems.

Since the embedded system is dedicated to specific tasks, design engineers can optimize it to reduce the size and cost of the product and increase the reliability and performance. Some embedded systems are mass-produced, benefiting from economies of scale.

Embedded systems range from portable devices such as digital watches and MP3 players, to large stationary installations like traffic lights, factory controllers, and largely complex systems like hybrid vehicles, MRI, and avionics. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large chassis or enclosure.

1.2 Examples:-

Some examples of embedded systems include ATMs, cell phones, printers, thermostats, calculators, and videogame consoles. Handheld computers or PDAs are also considered embedded devices because of the nature of their hardware design, even though they are more expandable in software terms**.**

1.3 Features of an Embedded System:-

1. They are small in size.
2. They use little power.
3. They are low cost.

Diagram of Embedded System:-

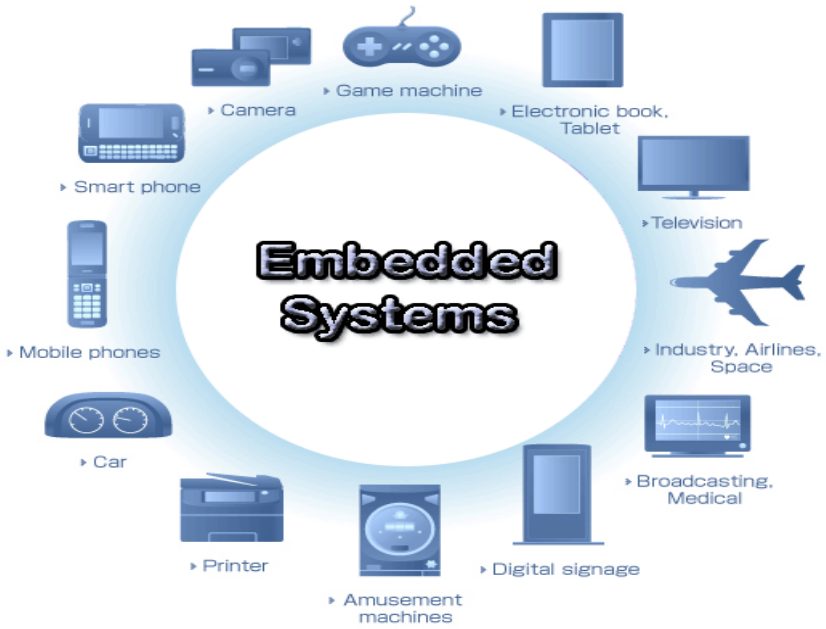


Fig no: 1.1

1.4 Components of an Embedded System:-

An embedded system has three main components : Hardware, Software and time operating system.

i) Hardware:-

1. Power Supply
2. Processor
3. Memory
4. Timers
5. Serial communication ports
6. Output/Output circuits
7. System application specific circuits

ii) Software:- The application software is required to perform the series of tasks.

An embedded system has software designed to keep in view of three constraints:

• Availability of System Memory

• Availability of processor speed

• The need to limit power dissipation when running the system continuously in cycles of wait for events, run , stop and wake up.

iii) Real Time Operating System: -(RTOS) It supervises the application software and provides a mechanism to let the processor run a process as per scheduling and do the switching from one process (task) to another process.

Source:- Internet.

Chapter: 2

HARDWARE REQUIREMENTS

HARDWARE COMPONENTS:

1. TRANSFORMER (230 – 12 V AC)
2. VOLTAGE REGULATOR (LM 7805)
3. RECTIFIER
4. FILTER
5. MICROCONTROLLER (AT89S52/AT89C51)
6. LED'S
7. MOTOR DRIVER(L293D)
8. DC MOTOR
9. BLUETOOTH MODULE
10. 1N4007
11. RESISTOR
12. CAPACITOR

2.1 BATTERY

An electrical battery is a combination of one or more electrochemical cells, used to convert stored chemical energy into electrical energy. The battery has become a common power source for many household and industrial applications.

Batteries may be used once and discarded, or recharged for years as in standby power applications. Miniature cells are used to power devices such as hearing aids and wristwatches; larger batteries provide standby power for telephone exchanges or computer data centers.

different chemistries but approximately the same emf of 1.5 volts; likewise NiCd and NiMH cells have different chemistries, but approximately the same emf of 1.2 volts. On the other hand the high electrochemical potential changes in the reactions of lithium compounds give lithium cells emfs of 3 volts or more.

2.2 VOLTAGE REGULATOR 7805

Features

- Output Current up to 1A.
- Output Voltages of 5, 6, 8, 9, 10, 12, 15, 18, 24V.
- Thermal Overload Protection.
- Short Circuit Protection.

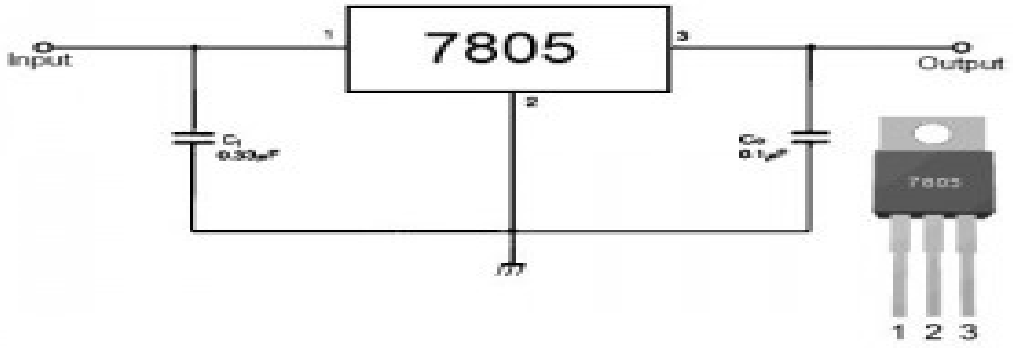


Fig no: 2.1

2.3 RECTIFIER

A rectifier is an electrical device that converts alternating current (AC), which periodically reverses direction, to direct current (DC), current that flows in only one direction, a process known as rectification. Rectifiers have many uses including as components of power supplies and as detectors of radio signals. Rectifiers may be made of solid state diodes, vacuum tube diodes, mercury arc valves, and other components. The output from the transformer is fed to the rectifier. It converts A.C. into pulsating D.C. The rectifier may be a half wave or a full wave rectifier. In this project, a bridge rectifier is used because of its merits like good stability and full wave rectification. In positive half cycle only two diodes(1 set of parallel diodes) will conduct, in negative half cycle remaining two diodes will conduct and they will conduct only in forward bias only.

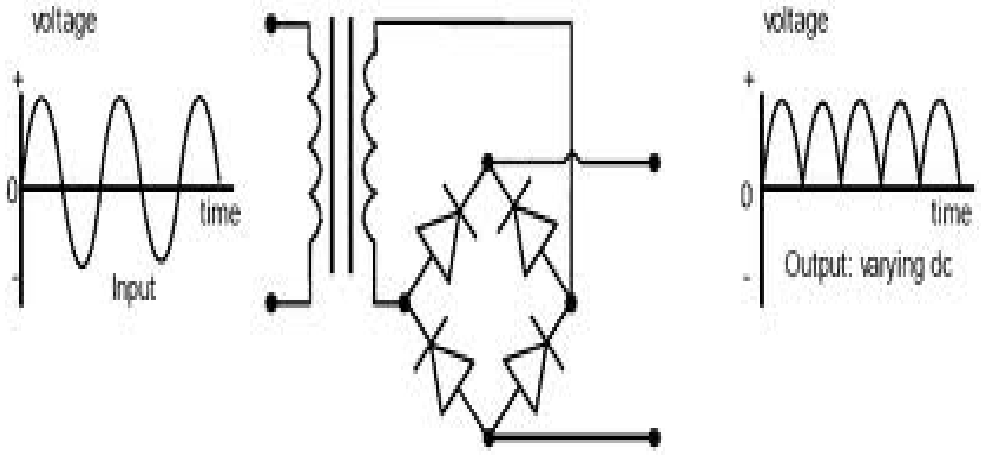


Fig no: 2.2

2.4 FILTER

Capacitive filter is used in this project. It removes the ripples from the output of rectifier and smoothens the D.C. Output received from this filter is constant until the mains voltage and load is maintained constant. However, if either of the two is varied, D.C. voltage received at this point changes. Therefore a regulator is applied at the output stage.

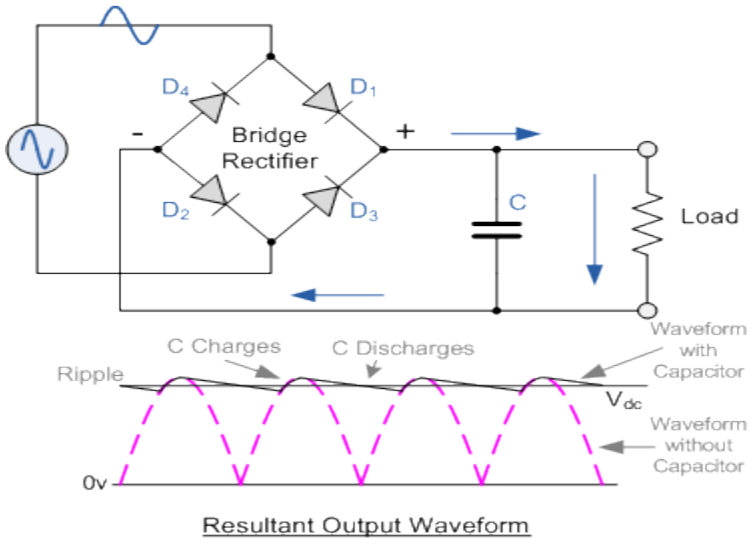


Fig: 2.3

2.5 MICROCONTROLLER AT89S52

The AT89S52 is a low-power, high-performance CMOS 8-bit microcontroller with 8K bytes of in-system programmable Flash memory. The device is manufactured using Atmel's high-density non volatile memory technology and is compatible with the industry standard 80C51 instruction set and pin out. The on-chip Flash allows the program memory to be reprogrammed in-system or by a conventional non volatile memory programmer. By combining a versatile 8-bit CPU with in-system programmable Flash on a monolithic chip, the Atmel AT89S52 is a powerful microcontroller which provides a highly-flexible and cost-effective solution to many embedded control applications.

Features:

• Compatible with MCS®-51 Products

• 8K Bytes of In-System Programmable (ISP) Flash Memory

– Endurance: 10,000 Write/Erase Cycles

• 4.0V to 5.5V Operating Range

• Fully Static Operation: 0 Hz to 33 MHz

• Three-level Program Memory Lock

• 256 x 8-bit Internal RAM

• 32 Programmable I/O Lines

• Three 16-bit Timer/Counters

• Eight Interrupt Sources

• Full Duplex UART Serial Channel

• Low-power Idle and Power-down Modes

Pin Configurations of AT89S52

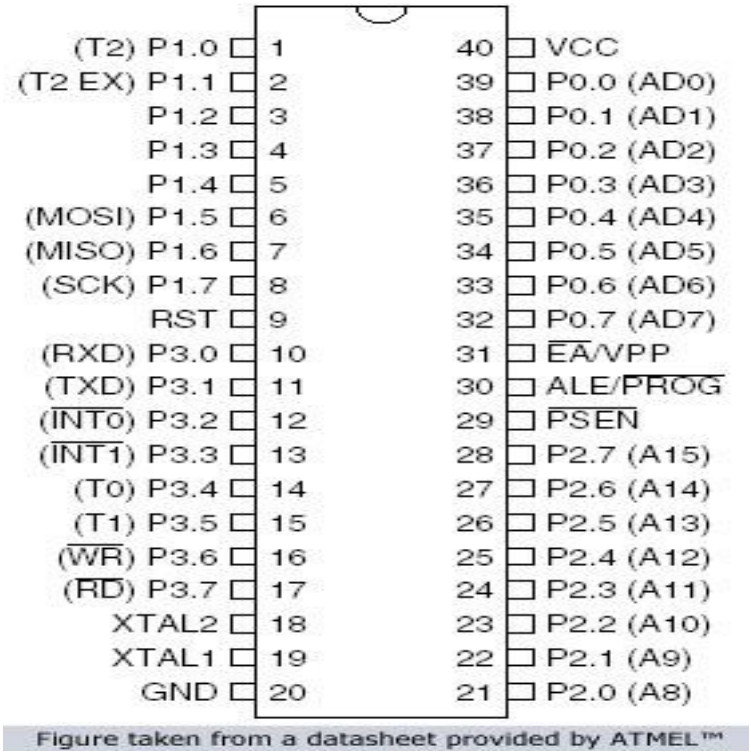


Fig no:2.4

Pin Description:

VCC:

Supply voltage.

GND:

Ground

Port 0:

Port 0 is an 8-bit open drain bidirectional I/O port. As an output port, each pin can sink eight TTL inputs. When 1s are written to port 0 pins, the pins can be used as high-impedance inputs. Port 0 can also be configured to be the multiplexed low-order address/data bus during accesses to external program and data memory. In this mode, P0 has internal pull-ups. Port 0 also receives the code bytes during Flash programming and outputs the code bytes during program verification. External pull-ups are required during program verification.

Port 1:

Port 1 is an 8-bit bidirectional I/O port with internal pull-ups. The Port 1 output buffers can sink/source four TTL inputs. When 1s are written to Port 1 pins, they are pulled high by the internal pull-ups and can be used as inputs. As inputs, Port 1 pins that are externally being pulled low will source current (IIL) because of the internal pull-ups. In addition, P1.0 and P1.1 can be configured to be the timer/counter 2 external count input (P1.0/T2) and the timer/counter 2 trigger input (P1.1/T2EX).

Port 2:

Port 2 is an 8-bit bidirectional I/O port with internal pull-ups. The Port 2 output buffers can sink/source four TTL inputs. When 1s are written to Port 2 pins, they are pulled high by the internal pull-ups and can be used as inputs. As inputs, Port 2 pins that are externally being pulled low will source current (IIL) because of the internal pull-ups. Port 2 emits the high-order address byte during fetches from external program memory and during accesses to external data memory that use 16-bit addresses (MOVX @ DPTR). In this application, Port 2 uses strong internal pull-ups when emitting 1s. During accesses to external data memory that use 8-bit addresses (MOVX @ RI), Port 2 emits the contents of the P2 Special Function Register.

Port 3:

Port 3 is an 8-bit bidirectional I/O port with internal pull-ups. The Port 3 output buffers can sink/source four TTL inputs. When 1s are written to Port 3 pins, they are pulled high by the internal pull-ups and can be used as inputs. As inputs, Port 3 pins that are externally being pulled low will source current (IIL) because of the pull-ups.

RST:

Reset input. A high on this pin for two machine cycles while the oscillator is running resets the device. This pin drives high for 98 oscillator periods after the Watchdog times out. The DISRTO bit in SFR AUXR (address 8EH) can be used to disable this feature. In the default state of bit DISRTO, the RESET HIGH out feature is enabled.

ALE/PROG:

Address Latch Enable (ALE) is an output pulse for latching the low byte of the address during accesses to external memory. This pin is also the program pulse input (PROG) during Flash programming.

In normal operation, ALE is emitted at a constant rate of 1/6 the oscillator frequency and may be used for external timing or clocking purposes. Note, however, that one ALE pulse is skipped during each access to external data memory.

PSEN:

Program Store Enable (PSEN) is the read strobe to external program memory. When the AT89S52 is executing code from external program memory, PSEN is activated twice each machine cycle, except that two PSEN activations are skipped during each access to external data memory.

EA/VPP:

External Access Enable. EA must be strapped to GND in order to enable the device to fetch code from external program memory locations starting at 0000H up to FFFFH. Note, however, that if lock bit 1 is programmed, EA will be internally latched on reset. EA should be strapped to VCC for internal program executions. This pin also receives the 12-volt programming enable voltage (VPP) during Flash programming.

XTAL1:

Input to the inverting oscillator amplifier and input to the internal clock operating circuit.

XTAL2: Output from the inverting oscillator amplifier

2.6 LED'S

A light-emitting diode (LED) is a semiconductor light source. LEDs are used as indicator lamps in many devices, and are increasingly used for lighting. When a light-emitting diode is forward biased (switched on), electrons are able to recombine with holes within the device, releasing energy in the form of photons. This effect is called electroluminescence and the color of the light (corresponding to the energy of the photon) is determined by the energy gap of the semiconductor.

Electronic Symbol:



Fig: 2.5

2.7 L293D

Features:

- Wide supply-voltage range: 4.5V to 36V
- Separate input- logic supply
- Internal ESD protection
- Thermal shutdown
- High-Noise-Immunity input

DESCRIPTION:

L293D is a dual H-bridge motor driver integrated circuit (IC). Motor drivers act as current amplifiers since they take a low-current control signal and provide a higher-current signal. This higher current signal is used to drive the motors.

Pin Diagram:

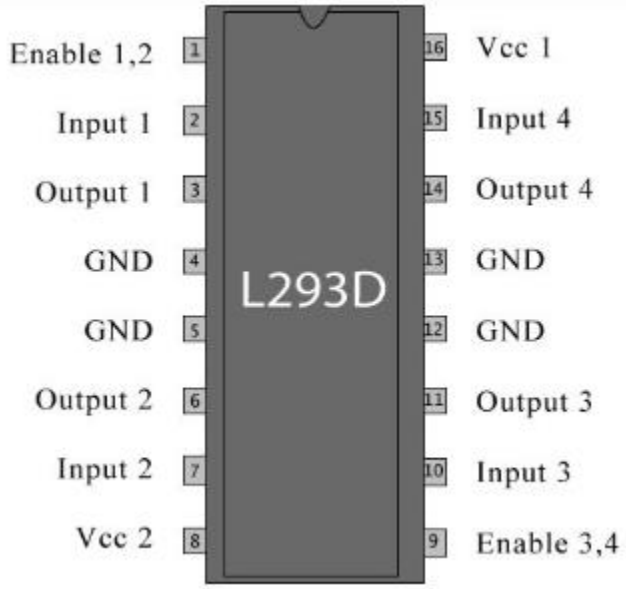


Fig no: 2.6

2.8 DC MOTOR

PRINCIPLES OF DC MOTOR OPERATION:

In any electric motor, operation is based on simple electromagnetism. A current-carrying conductor generates a magnetic field; when this is then placed in an external magnetic field, it will experience a force proportional to the current in the conductor, and to the strength of the external magnetic field. As you are well aware of from playing with magnets as a kid, opposite (North and South) polarities attract, while like polarities (North and North, South and South) repel. The internal configuration of a DC motor is designed to harness the magnetic interaction between a current-carrying conductor and an external magnetic field to generate rotational motion. Let's start by looking at a simple 2-pole DC electric motor (here red represents a magnet or winding with a "North" polarization, while green represents a magnet or winding with a "South" polarization).

Fig no: 2.7

Every DC motor has six basic parts -- axle, rotor (a.k.a., armature), stator, commutator, field magnet(s), and brushes. In most common DC motors, the external magnetic field is produced by high-strength permanent magnets[1]. The stator is the stationary part of the motor -- this includes the motor casing, as well as two or more permanent magnet pole pieces. The rotor rotates with respect to the stator. The rotor consists of windings (generally on a core), the windings being electrically connected to the commutator. The above diagram shows a common motor layout -- with the rotor inside the stator (field) magnets.

The geometry of the brushes, commutator contacts, and rotor windings are such that when power is applied, the polarities of the energized winding and the stator magnet(s) are

misaligned, and the rotor will rotate until it is almost aligned with the stator's field magnets. As the rotor reaches alignment, the brushes move to the next commutator contacts, and energize the next winding. Given our example two-pole motor, the rotation reverses the direction of current through the rotor winding, leading to a "flip" of the rotor's magnetic field, driving it to continue rotating.

Fig no: 2.8

In real life, though, DC motors will always have more than two poles (three is a very common number). In particular, this avoids "dead spots" in the commutator. You can imagine how with our example two-pole motor, if the rotor is exactly at the middle of its rotation (perfectly aligned with the field magnets), it will get "stuck" there. Meanwhile, with a two-pole motor, there is a moment where the commutator shorts out the power supply (i.e., both brushes touch both commutator contacts simultaneously). This would be bad for the power supply, waste energy, and damage motor components as well. Yet another disadvantage of such a simple motor is that it would exhibit a high amount of torque "ripple".

2.9 FACTORS THAT LED ANDROID TO BECOME WORLD'S MOST POPULAR OS

Android is open source and Google releases the code under the Apache License. This open-source code and permissive licensing allows the software to be freely modified and distributed by device manufacturers, wireless carriers and enthusiast developers. Additionally, Android has a large community of developers writing applications ("apps") that extend the functionality of devices, written primarily in a customized version of the Java programming language. In October 2012, there were approximately 700,000 apps available for Android, and the estimated number of applications downloaded from Google Play, Android's primary app store, was 25 billion. A developer survey conducted in April–May 2013 found that Android is the most popular platform for developers, used by 71% of the mobile developer population.

These factors have contributed towards making Android the world's most widely used Smartphone platform, overtaking Symbian in the fourth quarter of 2010, and the software of choice for technology companies who require a low-cost, customizable, lightweight operating system for high tech devices without developing one from scratch. As a result, despite being primarily designed for phones and tablets, it has seen additional applications on televisions, games consoles, digital cameras and other electronics. Android's open nature has further encouraged a large community of developers and enthusiasts to use the open-source code as a foundation for community-driven projects, which add new features for advanced users or bring Android to devices which were officially, released running other operating systems.

Android's share of the global Smartphone market, led by Samsung products, was 64% in March 2013. In July 2013 there were 11,868 models of Android device, scores of screen sizes and eight OS versions simultaneously in use. The operating system's success has made it a target for patent litigation as part of the so-called "Smartphone" between technology companies. As of May 2013, 48 billion apps have been installed from the Google Play store, and as of September 3, 2013, 1 billion Android devices have been activated.

Interface

Android's user interface is based on manipulation, using touch inputs that loosely correspond to real-world actions, like swiping, tapping, pinching and reverse pinching to manipulate on-screen objects. The response to user input is designed to be immediate and provides a fluid touch

interface, often using the vibration capabilities of the device to provide hap tic feedback to the user. Internal hardware such as accelerometers, gyroscopes and proximity sensors are used by some applications to respond to additional user actions, for example adjusting the screen from portrait to landscape depending on how the device is oriented, or allowing the user to steer a vehicle in a racing game by rotating the device, simulating control of a steering wheel.

Android devices boot to the home screen, the primary navigation and information point on the device, which is similar to the desktop found on PCs. Android home screens are typically made up of app icons and widgets; app icons launch the associated app, whereas widgets display live, auto-updating content such as the weather forecast, the user's email inbox, or a news ticker directly on the home screen. A home screen may be made up of several pages that the user can swipe back and forth between, though Android's home screen interface is heavily customizable, allowing the user to adjust the look and feel of the device to their tastes. Third party apps available on Google Play and other app stores can extensively re-theme the home screen, and even mimic the look of other operating systems, such as Windows Phone. Most manufacturers, and some wireless carriers, customize the look and feel of their Android devices to differentiate themselves from their competitors.

Present along the top of the screen is a status bar, showing information about the device and its connectivity. This status bar can be "pulled" down to reveal a notification screen where apps display important information or updates, such as a newly received email or SMS text, in a way that does not immediately interrupt or inconvenience the user. In early versions of Android these notifications could be tapped to open the relevant app, but recent updates have provided enhanced functionality, such as the ability to call a number back directly from the missed call notification without having to open the dialer app first. Notifications are persistent until read or dismissed by the user.

Android has a growing selection of third party applications, which can be acquired by users either through an app store such as Google Play or the Amazon Appstore, or by downloading and installing the application's APK file from a third-party site. The Play Store application allows users to browse, download and update apps published by Google and third-party developers, and is pre-installed on devices that comply with Google's compatibility requirements. The app filters the list of available applications to those that are compatible with the user's device, and

developers may restrict their applications to particular carriers or countries for business reasons. Purchases of unwanted applications can be refunded within 15 minutes of the time of download, and some carriers offer direct carrier billing for Google Play application purchases, where the cost of the application is added to the user's monthly bill. As of September 2012, there were more than 675,000 apps available for Android, and the estimated number of applications downloaded from the Play Store was 25 billion.

Applications are developed in the Java language using the Android software development kit (SDK). The SDK includes a comprehensive set of development tools, including a debugger, software libraries, a handset emulator based on QEMU, documentation, sample code, and tutorials. The officially supported integrated development environment (IDE) is Eclipse using the Android Development Tools (ADT) plug-in. Other development tools are available, including a Native Development Kit for applications or extensions in C or C++, Google App Inventor, a visual environment for novice programmers, and various cross platform mobile web applications frameworks

DEVELOPMENT OF ANDROID APPS

Android is developed in private by Google until the latest changes and updates are ready to be released, at which point the source code is made available publicly. This source code will only run without modification on select devices, usually the Nexus series of devices. With others, there are proprietary binaries which have to be provided by the manufacturer in order for Android to work. The green Android logo was designed by graphic designer Irina Blok.

Linux

Android consists of a kernel based on Linux kernel version 3.x (version 2.6 prior to Android 4.0 *Ice Cream Sandwich*), with middleware, libraries and APIs written in C, and application software running on an application framework which includes Java-compatible libraries based on Apache Harmony. Android uses the Dalvik virtual machine with just-in-time compilation to run Dalvik 'dex-code' (Dalvik Executable), which is usually translated from code. The main hardware platform for Android is the ARM architecture. There is support for x86from the Android-x86 project, and Google TV uses a special x86 version of Android. In 2013, free scale announced Android on its i.MX processor, i.MX5X and i.MX6X series. In 2012 Intel processors began to appear on more mainstream Android platforms, such as phones.

Android's Linux kernel has further architecture changes by Google outside the typical Linux kernel development cycle. Android does not have a native X Window System by default nor does it support the full set of standard GNU libraries, and this makes it difficult to port existing Linux applications or libraries to Android. Support for simple C and SDL applications is possible by injection of a small Java shim and usage of the JNI[74] like, for example, in the Jagged Alliance 2 port for Android.[75]

VERSIONS OF ANDROID-OS

This chart provides data about the relative number of devices accessing the Play Store recently and running a given version of the Android platform as of October 2, 2013.

Version	Code name	Release date	API level	Distribution
4.4	*Kit Kat*	TBA	TBA	0%
4.3	*Jelly Bean*	July 24, 2013	18	1.5%
4.2.x	*Jelly Bean*	November 13, 2012	17	10.6%
4.1.x	*Jelly Bean*	July 9, 2012	16	36.5%
4.0.3–4.0.4	*Ice Cream Sandwich*	December 16, 2011	15	20.6%
3.2	*Honeycomb*	July 15, 2011	13	0.1%
3.1	*Honeycomb*	May 10, 2011	12	0%

Version	Code name	Release date	API level	Distribution
2.3.3–2.3.7	*Gingerbread*	February 9, 2011	10	28.5%
2.3–2.3.2	*Gingerbread*	December 6, 2010	9	0%
2.2	*Froyo*	May 20, 2010	8	2.2%
2.0–2.1	*Eclair*	October 26, 2009	7	0%
1.6	*Donut*	September 15, 2009	4	0%
1.5	*Cupcake*	April 30, 2009	3	0%

ANDROID APPLICATION OPERATED BLUETOOTH

The Android platform includes support for the Bluetooth network stack, which allows a device to wirelessly exchange data with other Bluetooth devices. The application framework provides access to the Bluetooth functionality through the Android Bluetooth APIs. These APIs let applications wirelessly connect to other Bluetooth devices, enabling point-to-point and multipoint wireless features.

- Using the Bluetooth APIs, an Android application can perform the following:
- Scan for other Bluetooth devices
- Query the local Bluetooth adapter for paired Bluetooth devices
- Establish RFCOMM channels
- Connect to other devices through service discovery
- Transfer data to and from other devices
- Manage multiple connections

This document describes how to use *Classic Bluetooth*. Classic Bluetooth is the right choice for more battery-intensive operations such as streaming and communicating between Android devices. For Bluetooth devices with low power requirements, Android 4.3 (API Level 18) introduces API support for Bluetooth Low Energy. To learn more, see Bluetooth Low Energy.

This document describes how to use the Android Bluetooth APIs to accomplish the four major tasks necessary to communicate using Bluetooth: setting up Bluetooth, finding devices that are either paired or available in the local area, connecting devices, and transferring data between devices.

All of the Bluetooth APIs is available in the android.bluetooth package. Here's a summary of application permissions.

Finding Devices

you can find remote Bluetooth devices either through device discovery or by querying the list of paired (bonded) devices.Device discovery is a scanning procedure that searches the local area for Bluetooth enabled devices and then requesting some information about each one (this is sometimes referred to as "discovering," "inquiring" or "scanning"). However, a Bluetooth device within the local area will respond to a discovery request only if it is currently enabled to be discoverable. If a device is discoverable, it will respond to the discovery request by sharing some information, such as the device name, class, and its unique MAC address. Using this information, the device performing discovery can then choose to initiate a connection to the discovered device.

Once a connection is made with a remote device for the first time, a pairing request is automatically presented to the user. When a device is paired, the basic information about that device (such as the device name, class, and MAC address) is saved and can be read using the Bluetooth APIs. Using the known MAC address for a remote device, a connection can be initiated with it at any time without performing discovery (assuming the device is within range).

Remember there is a difference between being paired and being connected. To be paired means that two devices are aware of each other's existence, have a shared link-key that can be used for authentication, and are capable of establishing an encrypted connection with each other. To be connected means that the devices currently share an RFCOMM channel and are able to transmit data with each other. The current Android Bluetooth API's require devices to be paired before an RFCOMM connection can be established. (Pairing is automatically performed when you initiate an encrypted connection with the Bluetooth APIs.)

Discovering devices

To start discovering devices, simply call startDiscovery(). The process is asynchronous and the method will immediately return with a boolean indicating whether discovery has successfully started. The discovery process usually involves an inquiry scan of about 12 seconds, followed by a page scan of each found device to retrieve its Bluetooth name.

Your application must register a BroadcastReceiver for the ACTION_FOUND Intent in order to receive information about each device discovered. For each device, the system will broadcast the ACTION_FOUND Intent. This Intent carries the extra fields EXTRA_DEVICE and EXTRA_CLASS, containing a BluetoothDevice and aBluetoothClass,

Managing a Connection

When you have successfully connected two (or more) devices, each one will have a connectedBluetoothSocket. This is where the fun begins because you can share data between devices. Using theBluetoothSocket, the general procedure to transfer arbitrary data is simple:

1.Get the InputStream and OutputStream that handle transmissions through the socket, viagetInputStream() and getOutputStream(), respectively.Read and write data to the streams with read(byte[]) and write(byte[]).That's it.There are, of course, implementation details to consider. First and foremost, you should use a dedicated thread for all stream reading and writing. This is important because both read(byte[]) and write(byte[])methods are blocking calls. read(byte[]) will block until there is something to read from the stream.write(byte[]) does not usually block, but can block for flow control if the remote device is not callingread(byte[]) quickly enough and the intermediate buffers are full. So, your main loop in the thread should be dedicated to reading from the InputStream. A separate public method in the thread can be used to initiate writes to the OutputStream.

2.10 1N4007

Diodes are used to convert AC into DC these are used as half wave rectifier or full wave rectifier. Three points must he kept in mind while using any type of diode.

1.Maximum forward current capacity

2.Maximum reverse voltage capacity

3.Maximum forward voltage capacity

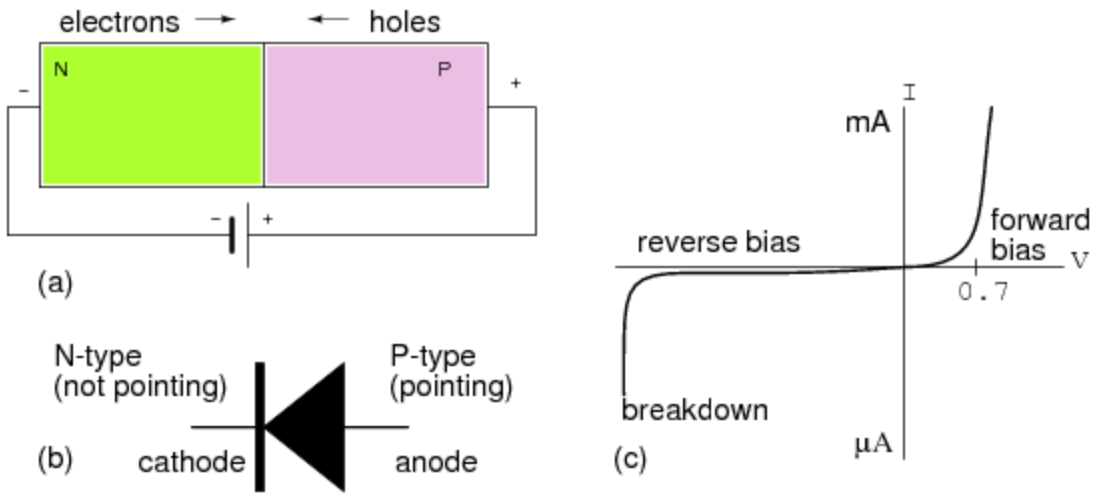


Fig no:2.9

2.11 RESISTORS

A resistor is a two-terminal electronic component designed to oppose an electric current by producing a voltage drop between its terminals in proportion to the current, that is, in accordance with Ohm's law:

$$V = IR$$



Fig no: 2.10

2.12 CAPACITORS

A capacitor or condenser is a passive electronic component consisting of a pair of conductors separated by a dielectric. When a voltage potential difference exists between the conductors, an electric field is present in the dielectric. This field stores energy and produces a mechanical force

between the plates. The effect is greatest between wide, flat, parallel, narrowly separated conductors.

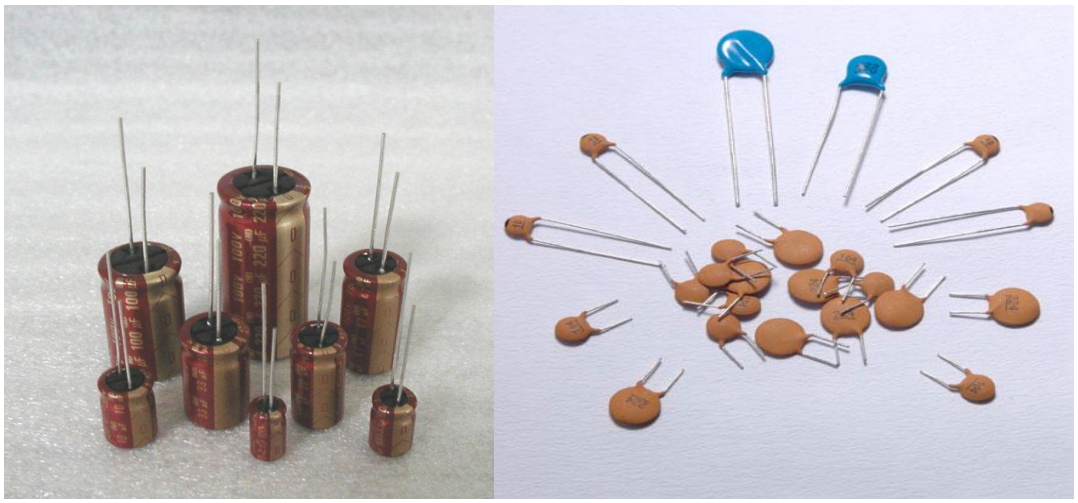


Fig no: 2.11

2.13 SOFTWARE REQUIREMENTS

INTRODUCTION TO KEIL MICRO VISION (IDE)

Keil an ARM Company makes C compilers, macro assemblers, real-time kernels, debuggers, simulators, integrated environments, evaluation boards, and emulators for ARM7/ARM9/Cortex-M3, XC16x/C16x/ST10, 251, and 8051 MCU families. Keil development tools for the 8051 Microcontroller Architecture support every level of software developer from the professional applications engineer to the student just learning about embedded software development. When starting a new project, simply select the microcontroller you use from the Device Database and the µVision IDE sets all compiler, assembler, linker, and memory options for you.Keil is a cross compiler. So first we have to understand the concept of compilers and cross compilers. After then we shall learn how to work with keil.

CONCEPT OF COMPILER

Compilers are programs used to convert a High Level Language to object code. Desktop compilers produce an output object code for the underlying microprocessor, but not for other microprocessors. I.E the programs written in one of the HLL like ‘C’ will compile the code to

run on the system for a particular processor like x86 (underlying microprocessor in the computer). For example compilers for Dos platform is different from the Compilers for Unix platform So if one wants to define a compiler then compiler is a program that translates source code into object code. The compiler derives its name from the way it works, looking at the entire piece of source code and collecting and reorganizing the instruction. See there is a bit little difference between compiler and an interpreter. Interpreter just interprets whole program at a time while compiler analyses and execute each line of source code in succession, without looking at the entire program. The advantage of interpreters is that they can execute a program immediately. Secondly programs produced by compilers run much faster than the same programs executed by an interpreter. However compilers require some time before an executable program emerges. Now as compilers translate source code into object code, which is unique for each type of computer, many compilers are available for the same language.

EMBEDDED C

Use of embedded processors in passenger cars, mobile phones, medical equipment, aerospace systems and defense systems is widespread, and even everyday domestic appliances such as dish washers, televisions, washing machines and video recorders now include at least one such device. Because most embedded projects have severe cost constraints, they tend to use low-cost processors like the 8051 family of devices considered in this book. These popular chips have very limited resources available most such devices have around 256 bytes (not megabytes!) of RAM, and the available processor power is around 1000 times less than that of a desktop processor. As a result, developing embedded software presents significant new challenges, even for experienced desktop programmers. If you have some programming experience - in C, C++ or Java - then this book and its accompanying CD will help make your move to the embedded world as quick and painless as possible.

Chapter: 3

PROJECT WORKING

3.1 SCHEMATIC DIAGRAM

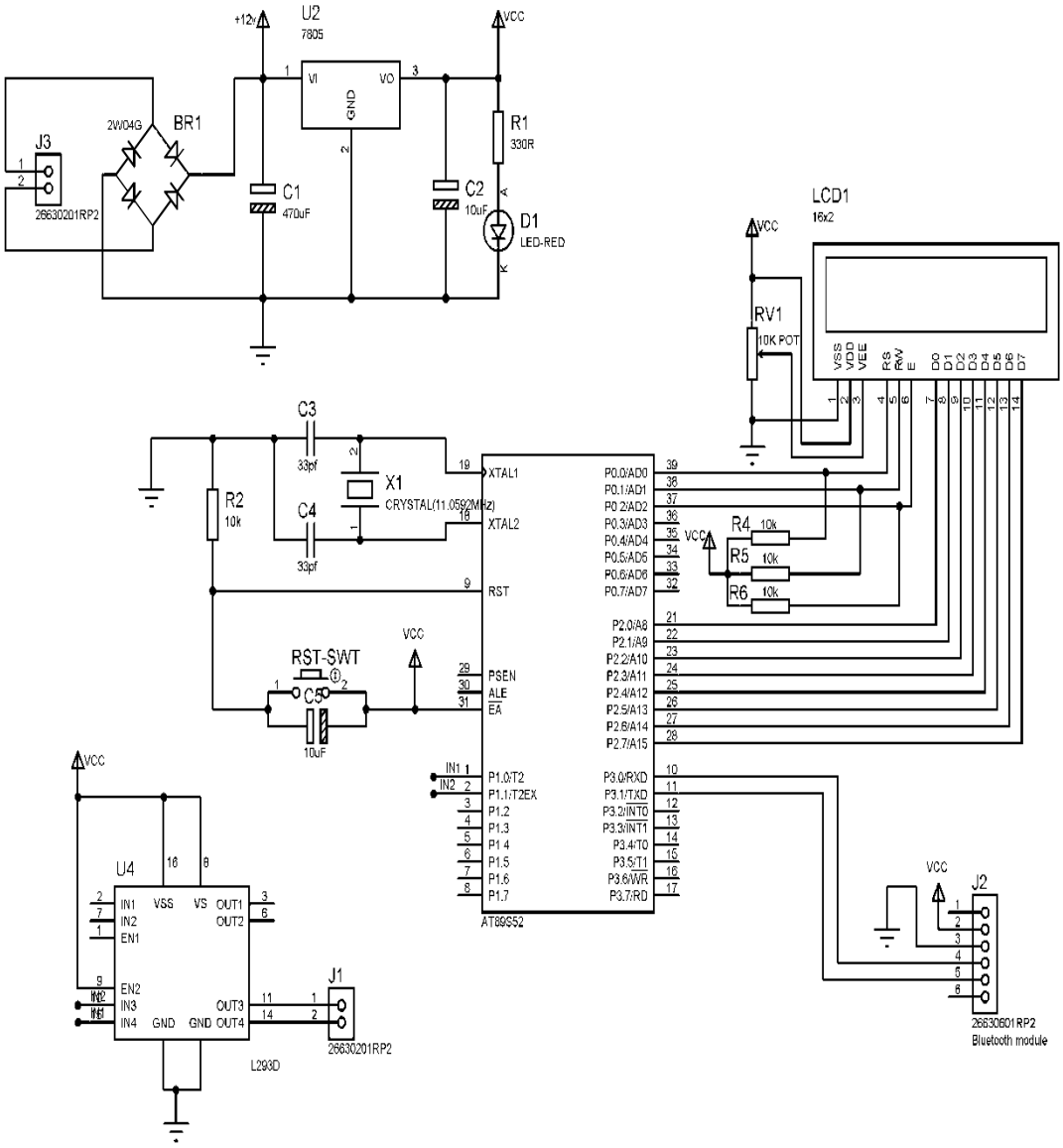


Fig no: 3.1

3.2 DESCRIPTION

POWER SUPPLY

The circuit uses standard power supply comprising of a step-down transformer from 230Vto 12V and 4 diodes forming a bridge rectifier that delivers pulsating dc which is then filtered by an electrolytic capacitor of about 470µF to 1000µF. The filtered dc being unregulated, IC LM7805 is used to get 5V DC constant at its pin no 3 irrespective of input DC varying from 7V to 15V. The input dc shall be varying in the event of input ac at 230volts section varies from 160V to 270V in the ratio of the transformer primary voltage V1 to secondary voltage V2 governed by the formula V1/V2=N1/N2. As N1/N2 i.e. no. of turns in the primary to the no. of turns in the secondary remains unchanged V2 is directly proportional to V1.Thus if the transformer delivers 12V at 220V input it will give 8.72V at 160V.Similarly at 270V it will give 14.72V.Thus the dc voltage at the input of the regulator changes from about 8V to 15V because of A.C voltage variation from 160V to 270V the regulator output will remain constant at 5V. The regulated 5V DC is further filtered by a small electrolytic capacitor of 10µF for any noise so generated by the circuit. One LED is connected of this 5V point in series with a current limiting resistor of 330Ω to the ground i.e., negative voltage to indicate 5V power supply availability. The unregulated 12V point is used for other applications as and when required.

STANDARD CONNECTIONS TO 8051 SERIES MICRO CONTROLLER

ATMEL series of 8051 family of micro controllers need certain standard connections. The actual number of the Microcontroller could be “89C51” , “89C52”, “89S51”, “89S52”, and as regards to 20 pin configuration a number of “89C2051”. The 4 set of I/O ports are used based on the project requirement. Every microcontroller requires a timing reference for its internal program execution therefore an oscillator needs to be functional with a desired frequency to obtain the timing reference as t =1/f. A crystal ranging from 2 to 20 MHz is required to be used at its pin number 18 and 19 for the internal oscillator. It may be noted here the crystal is not to be understood as crystal oscillator It is just a crystal, while connected to the appropriate pin of the microcontroller it results in oscillator function inside the microcontroller. Typically 11.0592 MHz crystal is used in general for most of the circuits using 8051 series microcontroller. Two

small value ceramic capacitors of 33pF each is used as a standard connection for the crystal as shown in the circuit diagram.

RESET

Pin no 9 is provided with an re-set arrangement by a combination of an electrolytic capacitor and a register forming RC time constant. At the time of switch on, the capacitor gets charged, and it behaves as a full short circuit from the positive to the pin number 9. After the capacitor gets fully charged the current stops flowing and pin number 9 goes low which is pulled down by a 10k resistor to the ground. This arrangement of reset at pin 9 going high initially and then to logic 0 i.e., low helps the program execution to start from the beginning. In absence of this the program execution could have taken place arbitrarily anywhere from the program cycle. A pushbutton switch is connected across the capacitor so that at any given time as desired it can be pressed such that it discharges the capacitor and while released the capacitor starts charging again and then pin number 9 goes to high and then back to low, to enable the program execution from the beginning. This operation of high to low of the reset pin takes place in fraction of a second as decided by the time constant R and C.

For example: A 10μF capacitor and a 10kΩ resistor would render a 100ms time to pin number 9 from logic high to low, there after the pin number 9 remains low.

External Access(EA):

Pin no 31 of 40 pin 8051 microcontroller termed as EA^{-} is required to be connected to 5V for accessing the program form the on-chip program memory. If it is connected to ground then the controller accesses the program from external memory. However as we are using the internal memory it is always connected to +5V.

MOTOR DRIVER IC L293D

L293D has 2 set of arrangements where one set has input 1, input 2, output 1 and output 2 and other set has input 3, input 4, output 3 and output 4, according to block diagram if pin no 2 & 7 are high then pin no 3 & 6 are also high. In this project only one set is been used.

If enable 1 and pin number 2 are high leaving pin number 7 as low then the motor rotates in forward direction.

If enable 1 and pin number 7 are high leaving pin number 2 as low then the motor rotates in reverse direction.

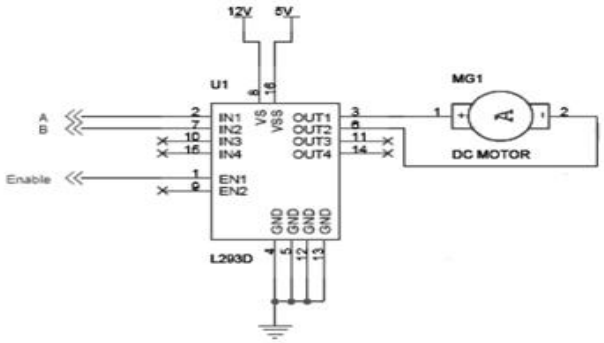


Fig no: 3.2

OPERATION EXPLANATION

When the circuit is powered on, the microcontroller initializes the LCD and serial communication interface. The LCD displays the project name and waits for input.

The system receives input through the serial interface connected to the Bluetooth receiver. The user first sends the * character to initiate password entry. The microcontroller then receives four password characters and compares them with the predefined password (1234) stored in the embedded C program.

If the password is incorrect, the LCD displays an invalid-key message. If the password is correct, the LCD displays a valid-key message and presents two available commands: # for opening and 0 for closing the door.

Upon receiving #, the microcontroller activates the L293D motor driver through P1.0 and P1.1 to rotate the DC motor in the opening direction. Upon receiving 0, it activates the motor in the opposite direction. After the programmed delay, both motor-control outputs are set to LOW, stopping the motor.

This demonstrates password-based access control using serial communication, embedded C programming and motor interfacing.

CONNECTIONS

The regulated 5 V supply is connected to pin 40 (VCC) of the 8051-family microcontroller, while ground is connected to pin 20 (GND).

The Bluetooth receiver communicates with the microcontroller through its serial interface. The LCD uses Port 2 for data transfer, while P0.0, P0.1 and P0.2 are assigned to the RS, RW and EN control signals, respectively, as defined in the embedded C program.

L293D Motor-Driver Connections:

The L293D motor-driver IC controls the direction of rotation of the DC motor. Microcontroller pin P1.0 is assigned to IN1 (pin 2), while P1.1 is assigned to IN2 (pin 7) of the L293D.

The DC motor is connected between OUT1 (pin 3) and OUT2 (pin 6). Pin 8 (VS) receives the 12 V motor supply, whereas pin 16 (VSS) receives the regulated 5 V logic supply. The ground terminals (pins 4, 5, 12 and 13) are connected to a common ground.

Enable pin EN1 (pin 1) must be maintained at a HIGH logic level to activate the motor-driver channel. The unused second channel remains disabled.

By changing the logic states of IN1 and IN2 through the embedded C program, the microcontroller controls the motor's direction of rotation, enabling the opening and closing operations of the door.

The integration of the power supply, Bluetooth communication module, microcontroller, LCD and L293D motor-driver circuit forms the hardware interface of the Android Password-Based Door Opener project.

3.3 **Project Diagram:**

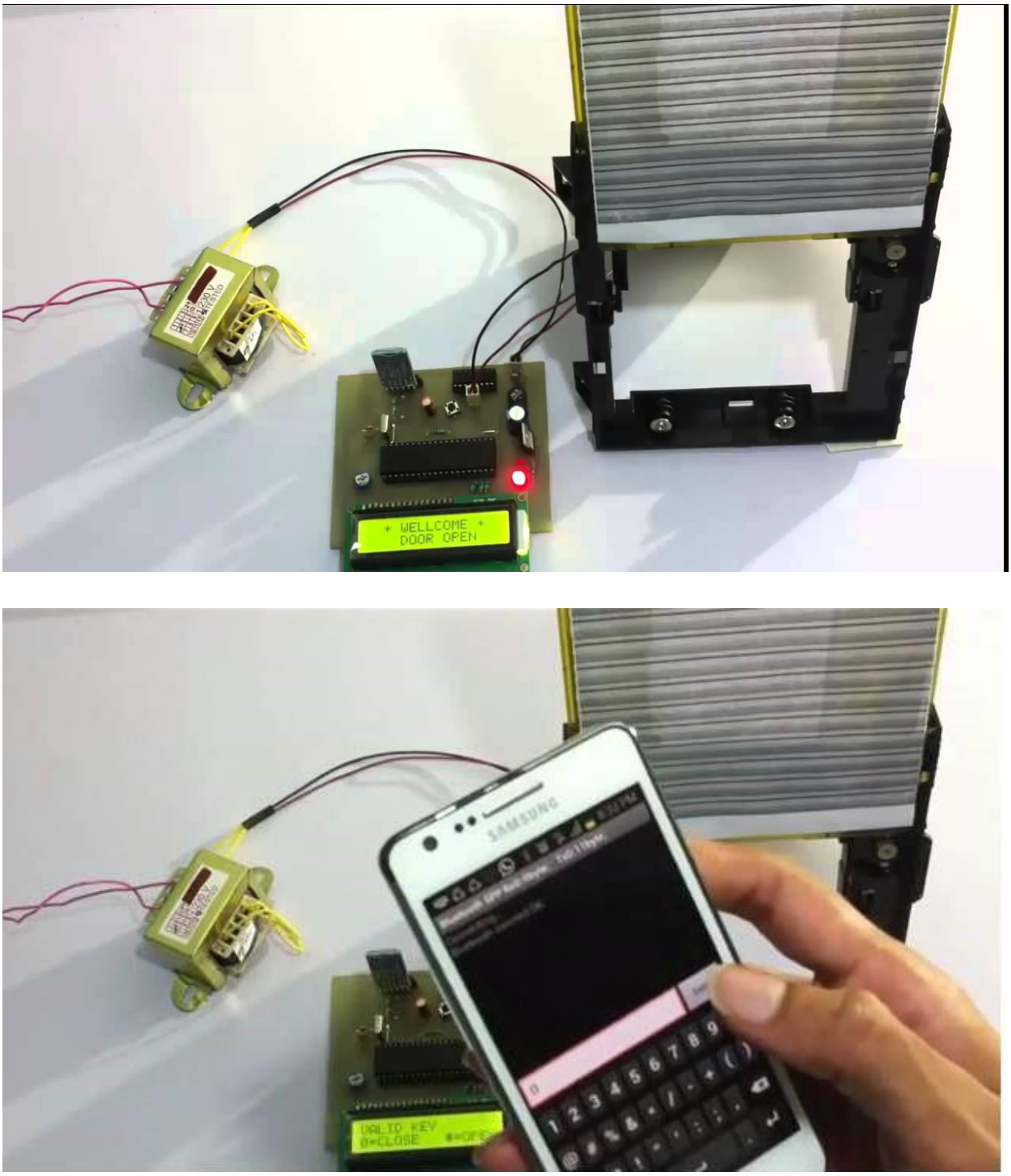


Fig no: 3.3

3.4 CODING

```
# include <at89x52.h>

# include <string.h>

# define LCDPort P2

sbit RS=P0^0;

sbit RW=P0^1;

sbit EN=P0^2;

sbit IN1=P1^0;

sbit IN2=P1^1;

unsigned char chr,ch;

unsigned char buff[80];

bit door_status,right_key,wrong_key,C=0;

unsigned int key_code;

idata unsigned char pass_word[5]={'1','2','3','4','\0'};

void toggle();

void main();

void init();

void init_display();
```

```
void delay(unsigned int delay_ms);

void lcd_cmd(unsigned char cmd);

void lcd_init();

void lcd_string(char *str);

void lcd_data(unsigned char data1);

void init_Serial();

unsigned char serial_read();

void TX_Char(unsigned char c);

void Rx_Char();

void Rx_Char2();

void main()

{

EA=1;

P0=0x00;

IN1=0;

IN2=0;

init();

while(1)
```

```
{
        Rx_Char();
        if(ch!='*')
        {
                lcd_cmd(0X01);
                lcd_cmd(0x80);
                lcd_string("  INVALID KEY ");
                delay(10);
        }
        else
        {
                lcd_cmd(0x01);
                lcd_cmd(0x80);
                lcd_string("ENTER 4 DIGIT");
                lcd_cmd(0xC0);
                lcd_string("PASSWORD");
                lcd_cmd(0xC8);
                delay(600);
```

```
Rx_Char2();
if(!strncmp(pass_word,buff,4))
{
key_code=0;
right_key=1;
wrong_key=0;
lcd_cmd(0x01);
lcd_cmd(0x80);
lcd_string("VALID KEY");
C=0;
lcd_cmd(0xC0);
lcd_string("0=CLOSE   #=OPEN");
while(!C)
{
Rx_Char();
 if(ch=='#')//#
{
```

```
door_status=1;
C=1;
lcd_cmd(0x01);
lcd_cmd(0x82);
lcd_string("* WELCOME *");
delay(6);
lcd_cmd(0xC4);
lcd_string("DOOR OPEN");
delay(10);
IN1=0;
IN2=1;
delay(900);
IN1=0;
IN2=0;
delay(10);
}
if(ch=='0')//0
{
```

```
door_status=0;

C=1;

delay(6);

lcd_cmd(0x01);

lcd_cmd(0x83);

lcd_string("**********");

delay(6);

lcd_cmd(0xC3);

lcd_string("DOOR CLOSE");

delay(10);

IN1=1;

IN2=0;

delay(900);

IN1=0;

IN2=0;

delay(10);

}

if(ch!='0' && ch!='#')
```

```
        {
                lcd_cmd(0X01);
                lcd_cmd(0x80);
                lcd_string("  INVALID KEY ");
                delay(10);
        }}}
else
        {
        key_code=0;
        right_key=1;
        wrong_key=0;
        lcd_cmd(0x01);
        lcd_cmd(0x83);
        lcd_string("INVALID KEY");
        }}}}
void init_Serial()
{
TMOD = 0x20;
```

```
SCON = 0x50;

TH1  = 0xFD;

TR1  = 1;

}

unsigned char serial_read()

{  while(!RI);

      RI = 0;

      return SBUF;}

void TX_Char(unsigned char c){

SBUF=c;

while(TI==0);

TI=0;

}void Rx_Char()

{

while(RI==0);

ch=SBUF;

RI=0;

}
```

```
void init()
{
        lcd_init();
        init_display();
        init_Serial();
}
void init_display()
        {
        lcd_cmd(0x01);
                lcd_cmd(0x82);
                lcd_string("ANDROID BASED ");
                lcd_cmd(0xC0);
                lcd_string("   DOOR OPENER   ");
                delay(4000);
        }
void delay(unsigned int delay_ms)
{
```

```
        unsigned int count,i;
for(count = 0;count<delay_ms;count++)
        {for(i =0;i < 120;i++);
        }
        delay_ms--; }
void lcd_string(char *str)
{       int i;
        for(i=0;str[i]!='\0';i++) lcd_data(str[i]);
}
void lcd_init()
        {  lcd_cmd(0x01);
                delay(10);
                lcd_cmd(0x06);
                delay(10);
                lcd_cmd(0x0c);
                delay(10);
                lcd_cmd(0x38);
                delay(10);
```

```
        }
void Rx_Char2()
{ unsigned char temp=0,i=0,h=1;
 while(h!=5)
{ Rx_Char();
                temp=ch;
                buff[i++]=temp;
                h++;
        }
        buff[i]='\0';
}
```

3.5 HARDWARE TESTING

CONTINUITY TEST: In electronics, a continuity test is the checking of an electric circuit to see if current flows (that it is in fact a complete circuit). A continuity test is performed by placing a small voltage (wired in series with an LED or noise-producing component such as a piezoelectric speaker) across the chosen path. If electron flow is inhibited by broken conductors, damaged components, or excessive resistance, the circuit is "open". Devices that can be used to perform continuity tests include multi meters which measure current and specialized continuity testers which are cheaper, more basic devices, generally with a simple light bulb that lights up when current flows.

An important application is the continuity test of a bundle of wires so as to find the two ends belonging to a particular one of these wires; there will be a negligible resistance between the "right" ends, and only between the "right" ends.

This test is the performed just after the hardware soldering and configuration has been completed. This test aims at finding any electrical open paths in the circuit after the soldering. Many a times, the electrical continuity in the circuit is lost due to improper soldering, wrong and rough handling of the PCB, improper usage of the soldering iron, component failures and presence of bugs in the circuit diagram. We use a multi meter to perform this test. We keep the multi meter in buzzer mode and connect the ground terminal of the multi meter to the ground. We connect both the terminals across the path that needs to be checked. If there is continuation then you will hear the beep sound.

POWER ON TEST:

This test is performed to check whether the voltage at different terminals is according to the requirement or not. We take a multi meter and put it in voltage mode. Remember that this test is performed without microcontroller. Firstly, we check the output of the transformer, whether we get the required 12 v AC voltage.

Then we apply this voltage to the power supply circuit. Note that we do this test without microcontroller because if there is any excessive voltage, this may lead to damaging the controller. We check for the input to the voltage regulator i.e., are we getting an input of 12v and an output of 5v. This 5v output is given to the microcontrollers' 40th pin. Hence we check for the voltage level at 40th pin. Similarly, we check for the other terminals for the required voltage. In this way we can assure that the voltage at all the terminals is as per the requirement.

CONCLUSION

The Android Password-Based Door Opener project demonstrates the integration of embedded programming, wireless communication and electronic motor control.

The system uses an 8051-family microcontroller to process password and door-control commands received through a Bluetooth serial interface. The embedded C program validates a predefined four-digit password and controls a DC motor through an L293D motor-driver circuit. An LCD provides information about password validation and door status.

The project provided practical experience in microcontroller programming, serial communication, hardware-software interfacing and the implementation of an embedded access-control prototype.